Bases conceptuales acerca del lenguaje unificado de modelado y patrones de diseño.
GA4-220501095-AA2-EV03.

Presentado por

Yésica Álvarez Sánchez



Servicio Nacional de Aprendizaje SENA
Análisis y Desarrollo de Software

Introducción

En el desarrollo de software es fundamental contar con herramientas que permitan entender y organizar un sistema antes de construirlo. Por eso, el UML se ha convertido en una guía importante para analizar, diseñar y comunicar cómo debe funcionar una aplicación. A través de diferentes diagramas, este lenguaje nos ayuda a visualizar procesos, identificar actores, definir estructuras y representar el comportamiento del sistema de una forma clara y ordenada. Gracias a esto, los equipos pueden trabajar con una visión común, evitar errores desde las primeras etapas y documentar todo el proyecto de manera más comprensible. En este trabajo profundizo en los conceptos, la utilidad y los tipos de diagramas UML para comprender mejor cómo apoyan el diseño de cualquier sistema.

Bases Conceptuales de UML

El UML es un lenguaje gráfico que utilizamos para modelar un sistema de forma visual. Su propósito principal no es programar, sino especificar, construir, visualizar y documentar

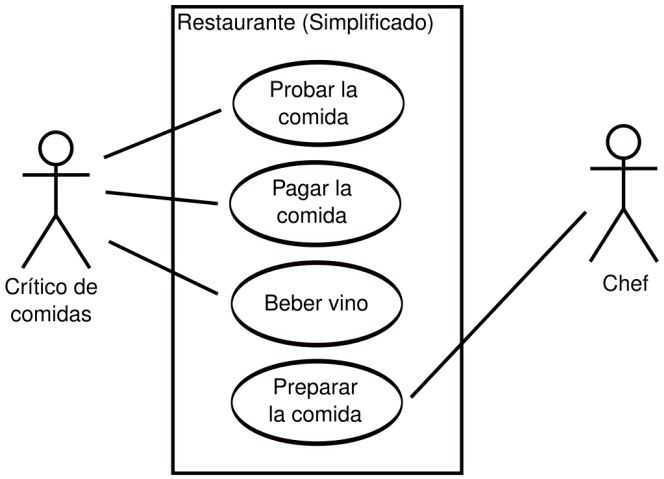
cualquier tipo de sistema, desde software hasta procesos de negocio. Gracias a UML podemos entender cómo funciona un sistema antes de escribir una sola línea de código. Se llama “Unificado” porque reúne y estandariza las mejores prácticas de varios métodos de modelado que existían anteriormente, trabajo que fue desarrollado por sus creadores principales, conocidos como los “Tres Amigos”: Grady Booch, Ivar Jacobson y James Rumbaugh.

La **importancia y utilidad de UML** radica en que permite una comunicación clara entre todos los involucrados en un proyecto, ya que traduce ideas complejas del cliente o del desarrollador a diagramas estandarizados que cualquiera puede entender. También es una herramienta fundamental para el análisis y diseño, porque ayuda a detectar errores, cuellos de botella o funcionalidades faltantes antes de llegar a la etapa de programación, donde corregirlos resulta más costoso. Además, los diagramas UML funcionan como una documentación clara y legible del sistema, facilitando su comprensión, mantenimiento y futuras mejoras.

Tipos de Diagramas UML

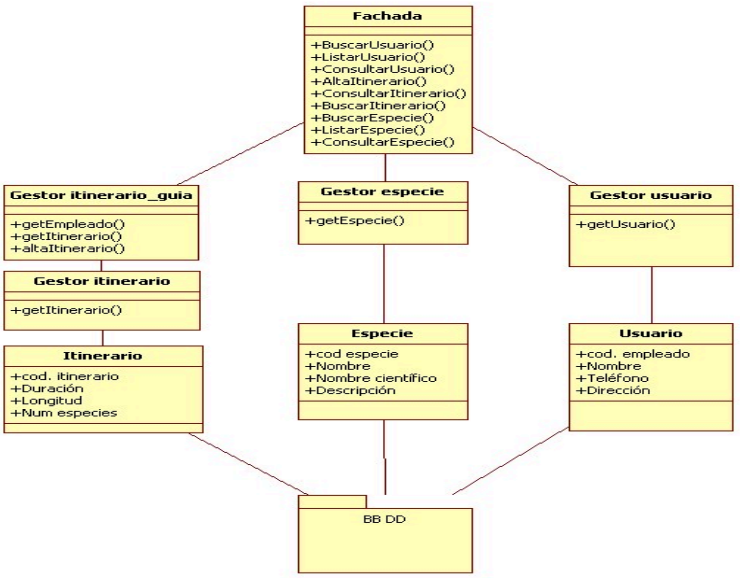
1. Diagrama de Casos de Uso

Es el diagrama que muestra qué puede hacer el sistema desde el punto de vista del usuario. Aquí aparecen los actores (personas o sistemas externos) y los casos de uso, que son las funciones principales. Sirve para entender los requisitos y qué espera cada usuario del sistema.



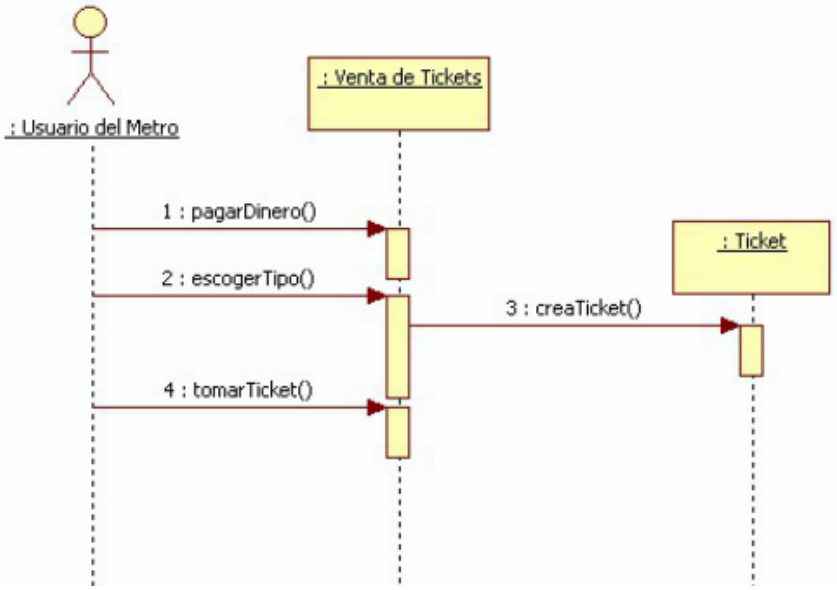
2. Diagrama de Clases

Muestra la estructura interna del sistema, es decir, las clases, sus atributos, métodos y cómo se relacionan entre sí. Es como el esqueleto del software. Se usa para diseñar la lógica y el modelo de datos.



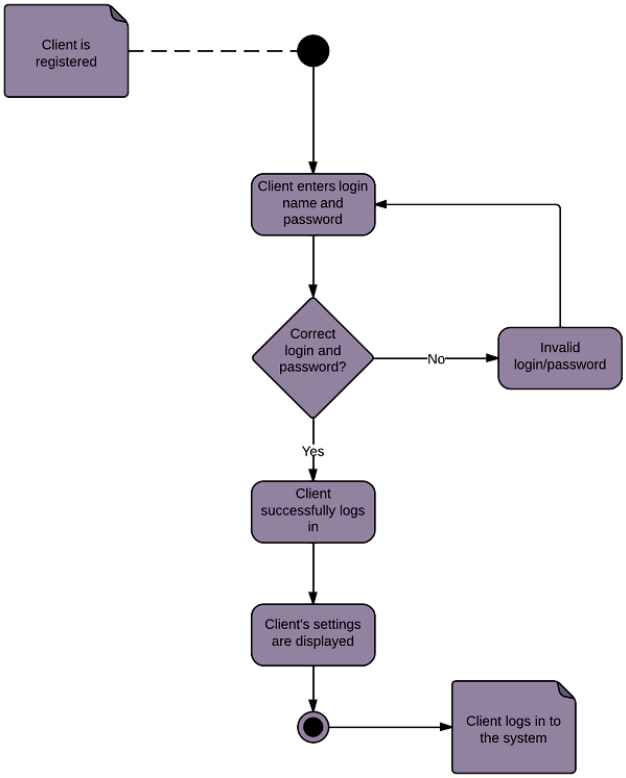
3. Diagrama de Secuencia

Explica cómo interactúan los objetos entre sí en un orden de tiempo. Muestra los mensajes que se envían unos a otros para completar una funcionalidad. Es útil para entender paso a paso cómo funciona un proceso.



4. Diagrama de Actividad

Representa el flujo de actividades o pasos dentro de un proceso. Se parece a un flujograma. Es ideal para mostrar decisiones, caminos alternos y tareas que se hacen en paralelo.

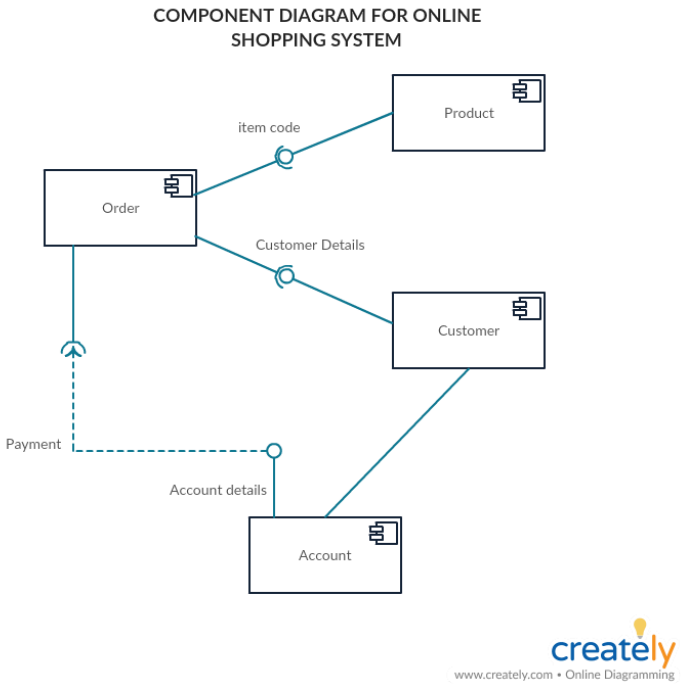


5. Diagrama de Estados

Muestra los estados por los que pasa un objeto y cómo cambia debido a eventos. Ejemplo: un paquete puede estar “Registrado”, luego “En tránsito” y finalmente “Entregado”. Ayuda cuando un objeto se comporta diferente según su estado.

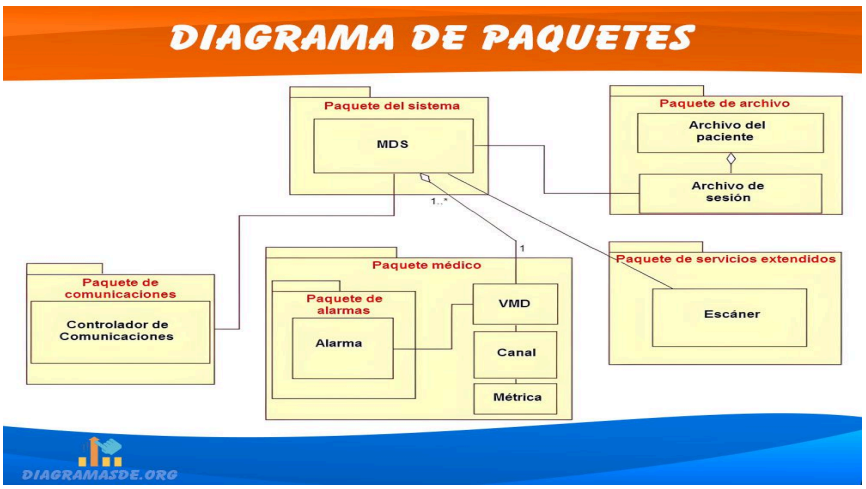
6. Diagrama de Componentes

Describe cómo está dividido el sistema en módulos o partes más grandes, como si fueran piezas de un rompecabezas. Cada componente tiene una función específica y se conecta con otros. Es útil cuando se diseña la arquitectura.



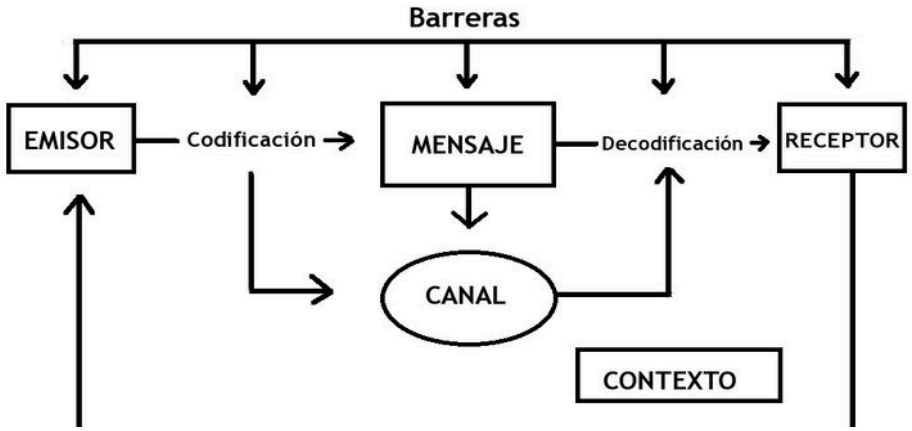
7. Diagrama de Paquetes

Agrupa clases o elementos del sistema en paquetes para mantener todo organizado. Es como crear carpetas que separan por módulos o áreas. Permite que el sistema sea más entendible y modular.



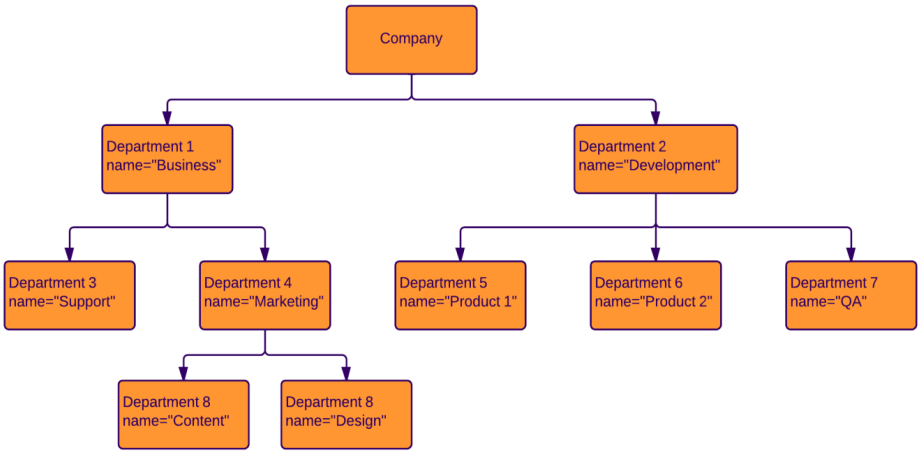
8. Diagrama de Comunicación

Similar al diagrama de secuencia, pero se enfoca más en quién se comunica con quién, en lugar del orden exacto del tiempo. Muestra las conexiones entre los objetos durante un proceso.



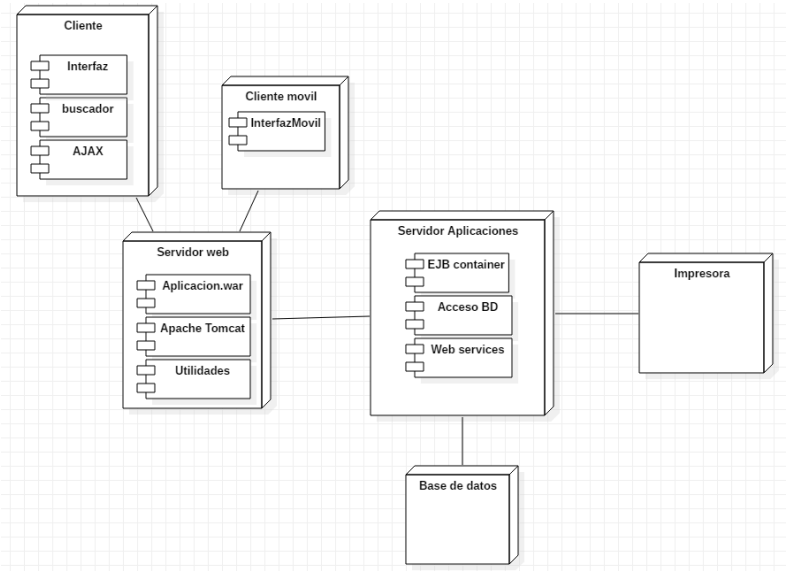
9. Diagrama de Objetos

Es una vista concreta del diagrama de clases, pero mostrando objetos reales con valores específicos. Útil para entender ejemplos reales del sistema en un momento exacto.



10. Diagrama de Despliegue

Muestra dónde se ejecuta el sistema físicamente, es decir, los servidores, dispositivos, nodos y cómo se conectan. Ayuda a entender cómo funciona el sistema en el mundo real.



Glosario ampliado

Actor:
Es cualquier persona o sistema externo que usa o se comunica con el sistema. Es quien realiza acciones o recibe resultados.

Caso de uso:
Es una función o acción que el sistema ofrece al actor. Representa algo que el usuario puede hacer o necesita que el sistema haga.

Clase:
Es como un molde o plantilla donde se definen las características (atributos) y las acciones (métodos) de los objetos que se van a crear.

Objeto:
Es un ejemplo real de una clase. Si la clase es el molde, el objeto es el producto que sale de ese molde con valores específicos.

Atributo:
Es un dato o característica que pertenece a una clase o a un objeto. Describe información que ese objeto debe almacenar.

Método (operación):
Es una acción o comportamiento que una clase puede realizar. Representa lo que un objeto sabe hacer.

Asociación:
Es una relación o conexión entre dos clases que muestra que de alguna manera están vinculadas o se necesitan mutuamente.

Multiplicidad:
Indica cuántos objetos de una clase pueden estar relacionados con objetos de otra clase. Por ejemplo, uno a muchos, uno a uno, muchos a muchos, etc.

Herencia (generalización):
Es una relación donde una clase hija toma las características de una clase padre. Representa un “es un”: por ejemplo, un *Perro* es un *Animal*.

Composición:
Es una relación fuerte donde una parte no puede existir sin el todo. Si el objeto principal se elimina, las partes también desaparecen.

Agregación:
Es una relación más suave donde el todo y las partes están conectados, pero las partes pueden existir por sí solas aunque el todo desaparezca.

Estado:
Es la condición o situación en la que se encuentra un objeto en un momento específico. Puede cambiar dependiendo de lo que ocurra.

Transición:
Es el cambio de un estado a otro provocado por un evento o una acción.

Estereotipo (stereotype):
Es una etiqueta especial que se usa para indicar que un elemento de UML cumple un rol particular o tiene un comportamiento específico.

Conclusión

El UML es una herramienta esencial en el desarrollo de software porque permite planear, organizar y comunicar cómo funcionará un sistema de forma visual. Sus diagramas ayudan a que los equipos entiendan mejor los procesos, las estructuras y las interacciones antes de construir el software. Usar UML mejora la claridad del proyecto, reduce errores y facilita la comunicación entre analistas, desarrolladores y demás participantes.